Model Diagnostic

The BF class can compute model diagnostics for a given model.

Lets consider the following model for a linear regression:

$$Y_i \sim \text{Normal}(\alpha + \beta X_i, \sigma)$$

$$\alpha \sim \text{Normal}(0, 1)$$

$$\beta \sim \text{Normal}(0, 1)$$

$$\sigma \sim \text{Uniform}(0, 50)$$

Python

```python
from BayesForge import bf
import jax.numpy as jnp

# setup platform-----------------------------------------------
m = bf(platform="cpu")

# import data -------------------------------------------------
data_path = m.load.howell1(only_path=True)
m.data(data_path, sep=";")
m.df = m.df[m.df.age > 18]
m.scale(data=["weight"])


# define model ------------------------------------------------
```

```python
def model(weight, height):
    a = m.dist.normal(178, 20, name="a")
    b = m.dist.log_normal(0, 1, name="b")
    s = m.dist.uniform(0, 50, name="s")
    m.dist.normal(a + b * weight, s, obs=height, shape=(weight.shape[0],))


# Run sampler ----------------------------------------------
m.fit(model, num_samples=500, num_chains=4)
m.summary()
```

```
bf v 0.0.48 package loaded
jax.local_device_count 32

  0%|          | 0/1500 [00:00<?, ?it/s]

  0%|          | 0/1500 [00:00<?, ?it/s]

  0%|          | 0/1500 [00:00<?, ?it/s]

  0%|          | 0/1500 [00:00<?, ?it/s]
```

	mean	sd	hdi_5.5%	hdi_94.5%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
a	154.65	0.28	154.20	155.07	0.01	0.01	1888.02	1337.52	1.0
b	5.82	0.27	5.39	6.25	0.01	0.01	2174.81	1461.49	1.0
s	5.15	0.20	4.81	5.45	0.00	0.00	2067.04	1201.14	1.0

R

```r
library(BayesForge)

# setup platform-------------------------------------------------
m <- importBF(platform = "cpu")

# import data ----------------------------------------------------
data_path <- m$load$howell1(only_path = TRUE)
m$data(data_path, sep = ";")
```

```
m$df <- m$df[m$df$age > 18, ]
m$scale(list("weight"))
m$data_to_model(list("weight", "height"))

# define model ------------------------------------------------
model <- function(weight, height) {
    a <- bf.dist.normal(178, 20, name = "a")
    b <- bf.dist.log_normal(0, 1, name = "b")
    s <- bf.dist.uniform(0, 50, name = "s")
    bf.dist.normal(a + b * weight, s, obs = height)
}

# Run sampler -------------------------------------------------
m$fit(model, num_samples = 500, num_chains = 4)
m$summary()
```

Julia

```
using PyBayesForge

# setup platform----------------------------------------------
m = importBF(platform = "cpu")

# import data -------------------------------------------------
data_path = m.load.howell1(only_path = true)
m.data(data_path, sep = ";")
m.df = m.df[m.df.age > 18]
m.scale(["weight"])

# define model ------------------------------------------------
@BF function model(weight, height)
    a = m.dist.normal(178, 20, name = "a")
    b = m.dist.log_normal(0, 1, name = "b")
    s = m.dist.uniform(0, 50, name = "s")
    m.dist.normal(a + b * weight, s, obs = height)
end

# Run sampler -------------------------------------------------
m.fit(model, num_samples = 500, num_chains = 4)
m.summary()
```

List of all available diagnostics

For additional documentation check the diagnostics API reference

Predictions from model based on specific data value

Python

```
m.sample() # Predictions from model base on data in data_on_model
m.sample(data=dict(weight=jnp.array([0.4])), remove_obs=False)# Predictions from a given valu
```

```
{'height': Array([[163.19593465],
        [147.48673429],
        [160.59276773],
        ...,
        [146.42503449],
        [150.87135473],
        [150.00865779]], dtype=float64)}
```

R

```
m$sample() # Predictions from model base on data in data_on_model
m$sample(data = list(weight = jnp$array(c(0.4))), remove_obs = FALSE) # Predictions from a gi
```

Julia

```
m.sample() # Predictions from model base on data in data_on_model
m.sample(data = pydict(weight = jnp.array([0.4])), remove_obs = false) # Predictions from a g
```

Forest plot of estimated values

Python

```
m.diag.forest()
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$forest()
```

Julia

```
@pyplot m.diag.forest()
```

Density plots of the posterior distribution

Python

```
m.diag.density()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$density()
```

Julia

```
@pyplot m.diag.density()
```

Posterior distribution plots

Python

```
m.diag.posterior()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$posterior()
```

Julia

```
@pyplot m.diag.posterior()
```

Trace plots for MCMC chains

Python

```
m.diag.plot_trace()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$plot_trace()
```

Julia

```
@pyplot m.diag.plot_trace()
```

Pairwise plots of the posterior distribution

Python

```
m.diag.pair()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$pair()
```

Julia

```
@pyplot m.diag.pair()
```

Plot autocorrelation of MCMC chains

Python

```
m.diag.autocor()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$autocor()
```

Julia

```
@pyplot m.diag.autocor()
```

Create rank plots for MCMC chains

Python

```
m.diag.rank()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$rank()
```

Julia

```
@pyplot m.diag.rank()
```

Evolution of effective sample size across iterations

Python

```
m.diag.plot_ess()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$plot_ess()
```

Julia

```
@pyplot m.diag.plot_ess()
```

HMC diagnostics: divergences and energy

Shows the E-BFMI energy statistic (marginal energy vs transition energy) to check for concentration problems, and flags divergent transitions per chain.

Python

```
m.diag.divergence_energy()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$divergence_energy()
```

Julia

```
@pyplot m.diag.divergence_energy()
```

Posterior predictive checks

All posterior predictive checks compare the observed outcome `y` with `yrep`, draws of the outcome simulated from the posterior (`m.diag.get_yrep()`); both are taken from the fitted model automatically. For count or categorical outcomes, `m.diag.ppc_bars()` and `m.diag.ppc_rootogram()` instead compare observed counts with replicated counts per category.

Density overlay

Overlays the KDE of the observed data (black) with n random draws from yrep (blue).

Python

```
m.diag.ppc_density()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_density()
```

Julia

```
@pyplot m.diag.ppc_density()
```

Histogram small multiples

Histograms of the observed data plus n random posterior predictive draws.

Python

```
m.diag.ppc_hist()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_hist()
```

Julia

```
@pyplot m.diag.ppc_hist()
```

Boxplots

Box-and-whisker plots of the observed data (black) versus n posterior predictive draws.

Python

```
m.diag.ppc_boxplot()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_boxplot()
```

Julia

```
@pyplot m.diag.ppc_boxplot()
```

Test statistic

Distribution of a test statistic (default: mean) over the posterior predictive draws, with the observed value marked and a Bayesian p-value.

Python

```
m.diag.ppc_stat()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_stat()
```

Julia

```
@pyplot m.diag.ppc_stat()
```

Predictive intervals

Inner and outer predictive intervals of yrep for each observation, with the observed y overlaid as scatter points.

Python

```
m.diag.ppc_intervals()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_intervals()
```

Julia

```
@pyplot m.diag.ppc_intervals()
```

Predictive ribbon

Continuous ribbon variant of the predictive intervals plot.

Python

```
m.diag.ppc_ribbon()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_ribbon()
```

Julia

```
@pyplot m.diag.ppc_ribbon()
```

Observed versus predictive mean

Scatter of observed y versus the posterior predictive mean E[yrep]; in a perfect model the points fall on the y = x diagonal.

Python

```
m.diag.ppc_error_scatter()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_error_scatter()
```

Julia

```
@pyplot m.diag.ppc_error_scatter()
```

Posterior residuals

Distribution of the mean residuals r_i = y_i - E[yrep_i].

Python

```
m.diag.ppc_error_hist()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_error_hist()
```

Julia

```
@pyplot m.diag.ppc_error_hist()
```

Observed versus individual replicates

Small-multiple scatterplots of the observed data against random individual rows of yrep.

Python

```
m.diag.ppc_scatter()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_scatter()
```

Julia

```
@pyplot m.diag.ppc_scatter()
```

PIT uniformity check

Density of the PIT values F(y_i) = P(yrep <= y_i), which are Uniform(0, 1) under a well-calibrated model; pass `log_likelihood=` to reweight the draws for the true leave-one-out version.

Python

```
m.diag.ppc_loo_pit()
```

```
/home/sosa/work/BF/BayesForge/Diagnostic/patch_diag.py:945: UserWarning:

ppc_loo_pit called without log_likelihood: computing a plain posterior-predictive PIT, not L(

Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_loo_pit()
```

Julia

```
@pyplot m.diag.ppc_loo_pit()
```

Predictive intervals sorted by y

Predictive intervals with the observations sorted by their observed value, which makes systematic over- and under-prediction easy to spot.

Python

```
m.diag.ppc_intervals_sorted()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_intervals_sorted()
```

Julia

```
@pyplot m.diag.ppc_intervals_sorted()
```

Regression plot

Observed data plus n smooth posterior regression lines E[y | x, theta] and their posterior mean, overlaid against a predictor variable.

Python

```
m.diag.plot_regression("weight", n = 20)
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$plot_regression("weight", n = 20L)
```

Julia

```
@pyplot m.diag.plot_regression("weight", n = 20)
```

Sensitivity

Prior sensitivity

Overlays the posteriors from models fitted with different priors to assess how much the conclusions change when the priors vary.

Python

```
# Refit the same model with a tighter prior on b
posteriors_weak = m.posteriors

def model_strong(weight, height):
    a = m.dist.normal(178, 20, name="a")
    b = m.dist.log_normal(0, 0.1, name="b")
    s = m.dist.uniform(0, 50, name="s")
    m.dist.normal(a + b * weight, s, obs=height, shape=(weight.shape[0],))

m.fit(model_strong, num_samples=500, num_chains=4)

m.diag.prior_sensitivity([posteriors_weak, m.posteriors],
                         labels=["weak prior", "strong prior"])
```

```
  0%|          | 0/1500 [00:00<?, ?it/s]

  0%|          | 0/1500 [00:00<?, ?it/s]

  0%|          | 0/1500 [00:00<?, ?it/s]
```

```
  0%|          | 0/1500 [00:00<?, ?it/s]
```

```
Unable to display output for mime type(s): text/html
```

R

```r
# Refit the same model with a tighter prior on b
posteriors_weak <- m$posteriors

model_strong <- function(weight, height) {
    a <- bf.dist.normal(178, 20, name = "a")
    b <- bf.dist.log_normal(0, 0.1, name = "b")
    s <- bf.dist.uniform(0, 50, name = "s")
    bf.dist.normal(a + b * weight, s, obs = height)
}

m$fit(model_strong, num_samples = 500, num_chains = 4)

m$diag$prior_sensitivity(list(posteriors_weak, m$posteriors),
                         labels = list("weak prior", "strong prior"))
```

Julia

```julia
# Refit the same model with a tighter prior on b
posteriors_weak = m.posteriors

@BF function model_strong(weight, height)
    a = m.dist.normal(178, 20, name = "a")
    b = m.dist.log_normal(0, 0.1, name = "b")
    s = m.dist.uniform(0, 50, name = "s")
    m.dist.normal(a + b * weight, s, obs = height)
end

m.fit(model_strong, num_samples = 500, num_chains = 4)

@pyplot m.diag.prior_sensitivity([posteriors_weak, m.posteriors],
                                 labels = ["weak prior", "strong prior"])
```

LOO influence

Scatter of the Pareto-k value of each observation; high k values ($k > 0.7$) indicate influential observations where the model is sensitive, potential outliers or model misspecification.

Python

```
m.diag.influence()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$influence()
```

Julia

```
@pyplot m.diag.influence()
```

Calibration

Empirical coverage of the predictive intervals versus their nominal credible interval levels; a well-calibrated model has empirical coverage close to nominal.

Python

```
m.diag.calibration()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$calibration()
```

Julia

```
@pyplot m.diag.calibration()
```

Multimodality check

Flags parameters whose posterior density has more than one local maximum (potential multimodality) and overlays the densities of the individual chains.

Python

```
m.diag.multimodality()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$multimodality()
```

Julia

```
@pyplot m.diag.multimodality()
```

Pareto-smoothed

Python

```
m.diag.loo()
```

```
Computed from 2000 posterior samples and 346 observations log-likelihood matrix.

         Estimate       SE
elpd_loo -1140.88    11.92
p_loo        2.44        -
```

R

```
m$diag$loo()
```

Julia

```
m.diag.loo()
```

Widely applicable information criterion

Python

```
m.diag.WAIC()
```

```
elpd_waic       -1140.877352
se                 11.916483
p_waic              2.434491
n_samples               2000
n_data_points            346
scale                    log
dtype: object
```

R

```
m$diag$WAIC()
```

Julia

```
m.diag.WAIC()
```